

Retrieval-Augmented Generation: A Primer

Retrieval-Augmented Generation, commonly abbreviated as RAG, is a technique for building question-answering systems that combines two components: a retrieval system that searches an external knowledge base, and a large language model that generates natural-language answers. Instead of relying purely on the knowledge a language model memorized during training, a RAG system looks up relevant information at the moment a question is asked and gives that information to the model as context.

This matters because large language models, no matter how large, cannot know about private documents, recent events, or specialized collections that were never part of their training data. A RAG system solves this by treating the language model as a reasoning and writing engine, while a separate retrieval component supplies the facts. The result is a system that can answer questions about a specific set of uploaded documents accurately, and which can openly admit when a document collection does not contain the answer, rather than inventing one.

The core idea of RAG was first popularized in a 2020 paper by researchers at Facebook AI Research, who showed that combining a neural retriever with a sequence-to-sequence generator produced more accurate and more verifiable answers than a generator alone. Since then, RAG has become one of the most widely used architectures for building practical, trustworthy AI applications on top of custom document collections, including academic research assistants, customer support tools, and enterprise search systems.

A defining property of a well-built RAG system is traceability: every answer can be traced back to the specific passages of text that were retrieved to produce it. This is what separates RAG from a simple chatbot - the system does not just produce a plausible-sounding answer, it produces an answer backed by citable evidence from the source documents.

The Stages of a RAG Pipeline

A typical RAG pipeline consists of several distinct stages, each of which can be tuned independently. The first stage is document ingestion, where files such as PDFs are uploaded and their raw text is extracted, along with structural metadata such as page numbers. Extracted text is then normalized: whitespace is collapsed, hyphenated line breaks are repaired, and other formatting artifacts introduced by the source format are cleaned up.

The second stage is chunking. Because embedding models and language models both have limited context windows, documents are split into smaller, overlapping segments called chunks. A good chunking strategy respects natural boundaries in the text, such as paragraphs and sentences, rather than cutting arbitrarily in the middle of a sentence. Overlap between consecutive chunks helps ensure that information near a chunk boundary is not lost. Chunk size and overlap are configurable parameters that directly affect retrieval quality: chunks that are too large dilute relevance, while chunks that are too small lose context.

The third stage is embedding generation. Each chunk of text is passed through an embedding model, which converts it into a dense numerical vector that captures its semantic meaning. Critically, the same embedding model must be used for both the document chunks and the user's query, so that semantically similar pieces of text end up close together in the same vector space.

The fourth stage is vector storage. The chunk embeddings, along with their source text and metadata, are stored in a vector database that supports efficient similarity search, such as PostgreSQL with the pgvector extension. When a user asks a question, the question is embedded using the same model, and the vector database returns the chunks whose embeddings are most similar to the question's embedding. This retrieval step is what grounds the system in the actual uploaded documents rather than the language model's general training knowledge.

Generation and Grounding

Once the most relevant chunks have been retrieved, they are assembled into a context block and passed to a large language model along with the user's original question and a carefully written system prompt. The system prompt instructs the model to answer strictly using the supplied context, to avoid introducing outside knowledge, and to explicitly say when the retrieved context does not contain enough information to answer the question.

This grounding step is the primary reason RAG systems hallucinate less than a language model used on its own. When a model is asked a question without any supporting context, it must rely entirely on patterns memorized during training, which can produce confident-sounding but incorrect answers, especially for niche or private information the model never saw during training. A RAG system instead constrains the model's attention to a small, relevant set of retrieved passages, and explicitly rewards the model for saying "I don't know" when those passages are insufficient, rather than guessing.

It is important to note that RAG substantially reduces hallucination but does not eliminate it entirely. A language model can still misinterpret retrieved text, combine facts from different chunks in a way that is not actually supported by either one, or occasionally fail to follow the grounding instructions in the system prompt. This is why answer evaluation for RAG systems typically measures faithfulness, sometimes called groundedness, separately from raw correctness: a faithful answer is one that is fully supported by the retrieved context, regardless of whether that context happens to be objectively true.

Well-designed RAG interfaces also display the retrieved source passages alongside the generated answer, with citations back to the originating document and page number. This lets a user independently verify the answer, which is especially important in academic, legal, and medical use cases where unverified AI-generated claims are unacceptable.

Evaluating a RAG System

Because a RAG system has two moving parts - retrieval and generation - evaluation typically measures each stage separately before evaluating the system end to end. Retrieval quality is commonly measured with metrics such as Hit Rate at K, which checks whether at least one relevant chunk appears among the top K retrieved results, Recall at K, which measures what fraction of all relevant chunks were retrieved, and Precision at K, which measures what fraction of the top K retrieved chunks were actually relevant.

Generation quality is harder to measure automatically, since there is rarely a single correct phrasing of a good answer. Two properties are typically assessed: correctness, meaning whether the answer matches the expected answer in substance, and faithfulness, meaning whether every claim in the answer is actually supported by the retrieved context rather than being invented. Faithfulness can be evaluated with a mix of automated methods, such as using a separate language model as a judge, and manual review by a human annotator, since automated judges are themselves imperfect and can be fooled by fluent but unsupported text.

A robust evaluation of a RAG system should use a small but carefully constructed dataset of questions with known expected answers and known source locations, so that both retrieval and generation can be checked against ground truth rather than judged subjectively. Experimenting with different configurations, such as varying the number of retrieved chunks (commonly written as the value of K), the chunk size, or the embedding model, and re-running the same evaluation dataset against each configuration, is the standard way to identify a good default configuration for a given document collection.

Limitations of this evaluation approach should always be documented alongside the results: automated LLM-based judges can have systematic biases, a small evaluation set may not generalize to all question types, and retrieval metrics do not capture whether the final generated answer was actually useful to a human reader.